

# DESOFS - GhostReport

2º Semestre

Phase 2: Sprint 1

1211551 – Alexandre Vieira

1250497 – Bárbara Silva

1250559 – Sofia Marques

## Índice

|                                                               |    |
|---------------------------------------------------------------|----|
| 1. Introdução .....                                           | 5  |
| 2. Arquitetura e Stack Tecnológico .....                      | 6  |
| 3. Autenticação e Validação JWT .....                         | 7  |
| 3.1 Estrutura do Token JWT .....                              | 7  |
| 3.2 Endpoint de Autenticação.....                             | 8  |
| 3.3 Configuração Stateless .....                              | 8  |
| 3.4 Endpoints Públicos e Protegidos .....                     | 9  |
| 3.5 Propriedades de Segurança da Autenticação .....           | 10 |
| 4. Controlo de Acessos Baseado em Papéis (RBAC) .....         | 10 |
| 4.1 Papéis .....                                              | 11 |
| 4.2 Aplicação das Regras RBAC.....                            | 11 |
| 4.3 Matriz de Acesso dos Controladores .....                  | 11 |
| 4.4 Regra de Propriedade dos Casos.....                       | 12 |
| 5. DTOs e Proteção Contra Mass Assignment .....               | 13 |
| 6. Validação e Sanitização de Entradas .....                  | 14 |
| 6.1 Bean Validation.....                                      | 14 |
| 6.2 Validação de Primitivas de Domínio .....                  | 15 |
| 6.3 Proteção Contra Path Traversal e Uploads Maliciosos ..... | 16 |
| 6.4 Tipos de Ficheiro Permitidos .....                        | 17 |
| 6.5 Limitações da Validação de Upload .....                   | 17 |
| 6.6 Impacto de Segurança.....                                 | 18 |
| 7. Tracking Code, Anonimato e Proteção Contra Abuso .....     | 18 |
| 7.1 Proteção Contra Enumeração.....                           | 19 |
| 7.2 Rate Limiting nos Fluxos Públicos .....                   | 19 |
| 7.3 Impacto de Segurança.....                                 | 20 |
| 7.4 Limitações Atuais .....                                   | 20 |
| 8. Error Handling e Prevenção de Information Disclosure.....  | 21 |
| 8.1 Estrutura das Respostas de Erro .....                     | 21 |
| 8.2 Mapeamento de Exceções.....                               | 22 |
| 8.3 Prevenção de Information Disclosure .....                 | 22 |
| 8.4 Integração com o Modelo de Segurança .....                | 23 |

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| 8.5 Exemplos de Erros Controlados .....                                     | 23 |
| 9. Auditoria e Monitorização de Segurança .....                             | 24 |
| 10. Pacotes de Evidência e Integridade de Backups.....                      | 25 |
| 10.1 Pacotes de Evidência .....                                             | 26 |
| 10.2 Verificação de Pacotes de Evidência .....                              | 26 |
| 10.3 Controlo de Acesso aos Pacotes .....                                   | 27 |
| 10.4 Backups da Aplicação .....                                             | 27 |
| 10.5 Manifest e Hashes de Integridade.....                                  | 28 |
| 10.6 Verificação de Backups .....                                           | 29 |
| 10.7 Restore Staging .....                                                  | 29 |
| 10.8 Proteções Contra Path Traversal e ZIP Slip .....                       | 30 |
| 10.9 Alertas e Auditoria .....                                              | 30 |
| 10.10 Limitações Atuais.....                                                | 31 |
| 10.11 Relação com Requisitos de Segurança .....                             | 31 |
| 11. Pipeline DevSecOps .....                                                | 31 |
| 11.1 Etapas da Pipeline.....                                                | 32 |
| 11.2 Base de Dados em Ambiente CI .....                                     | 33 |
| 11.3 Build e Testes Automatizados .....                                     | 33 |
| 11.4 SAST com SpotBugs .....                                                | 34 |
| 11.5 SCA com OWASP Dependency-Check.....                                    | 34 |
| 11.6 Secret Scanning com Gitleaks.....                                      | 35 |
| 11.7 DAST com OWASP ZAP Baseline.....                                       | 35 |
| 11.8 Artefactos de Segurança.....                                           | 36 |
| 11.9 Relação com SSDLC.....                                                 | 37 |
| 11.10 Limitações Atuais e Melhorias Futuras.....                            | 37 |
| 12. Threat Modeling: Ameaças Identificadas e Mitigações Implementadas ..... | 37 |
| 13. Testes Automatizados, Segurança e Cobertura .....                       | 39 |
| 13.1 Objetivo.....                                                          | 39 |
| 13.2 Estratégia de Testes .....                                             | 40 |
| 13.3 Testes Unitários .....                                                 | 40 |
| 13.4 Testes de Integração .....                                             | 41 |
| 13.5 Áreas de Segurança Validadas .....                                     | 41 |

|                                            |    |
|--------------------------------------------|----|
| 13.6 Testes aos Pacotes de Evidência ..... | 42 |
| 13.7 Execução dos Testes .....             | 43 |
| 13.8 Cobertura com JaCoCo .....            | 43 |
| 13.9 Execução em Pipeline .....            | 44 |
| 14. Conclusão .....                        | 46 |

## Índice de Figuras

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Figura 1 - Exemplo de resposta de autenticação .....                 | 8  |
| Figura 2 - Execução dos testes automatizados com Maven Wrapper ..... | 43 |
| Figura 3 - Relatório criado pelo JaCoCo .....                        | 44 |
| Figura 4 - Relatório criado pelo Pit .....                           | 44 |

## 1. Introdução

O projeto GhostReport foi desenvolvido com o objetivo de disponibilizar uma plataforma segura para submissão e tratamento de denúncias anônimas, garantindo a proteção da identidade do denunciante, a integridade das evidências digitais e a rastreabilidade das operações realizadas no sistema.

No âmbito da Phase 2 – Sprint 1, o principal foco do projeto incidiu na implementação prática de mecanismos de segurança aplicados tanto ao fluxo público de denúncias como às operações internas de backoffice. Esta sprint representou uma evolução relativamente à Phase 1, passando de uma abordagem centrada no desenho e análise de requisitos para uma implementação concreta dos controlos de segurança identificados anteriormente.

Durante esta fase foram implementados mecanismos de autenticação JWT stateless, controlo de acessos baseado em papéis (RBAC), validação e sanitização de inputs, proteção contra uploads maliciosos, auditoria de eventos de segurança, verificação de integridade de backups e geração segura de pacotes de evidência digital.

O sistema foi desenvolvido utilizando Spring Boot, Spring Security, PostgreSQL e autenticação JWT Bearer, sendo também integrados mecanismos de monitorização e testes automatizados de segurança para validação contínua dos controlos implementados.

Adicionalmente, o projeto seguiu princípios de secure-by-design, aplicando o princípio do menor privilégio, separação de responsabilidades e defesa em profundidade, com o objetivo de reduzir a superfície de ataque e aumentar a resiliência da aplicação perante ameaças comuns em aplicações web modernas.

## 2. Arquitetura e Stack Tecnológico

O GhostReport encontra-se estruturado segundo uma arquitetura cliente-servidor, separando o frontend público da lógica de negócio e persistência de dados do backend.

A interface frontend foi desenvolvida utilizando HTML, CSS e JavaScript estático, sendo responsável pela submissão de denúncias, acompanhamento através de tracking code e interação com os endpoints REST disponibilizados pelo backend.

O backend foi desenvolvido em Spring Boot, expondo uma API REST responsável pela autenticação, gestão de denúncias, controlo de acessos, geração de pacotes de evidência e operações administrativas e de auditoria.

A persistência de dados é realizada através de PostgreSQL, enquanto os ficheiros enviados pelos utilizadores são armazenados num sistema de armazenamento local controlado pela aplicação. Adicionalmente, o sistema inclui mecanismos de backup e verificação de integridade através de hashes SHA-256.

A autenticação da área de backoffice utiliza JWT Bearer tokens, sendo os acessos protegidos controlados através de Spring Security e regras RBAC.

| <b>Camada</b>                  | <b>Tecnologia</b>                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------|
| <i>Linguagem</i>               | Java 17                                                                                                          |
| <i>Framework</i>               | Spring Boot                                                                                                      |
| <i>Autenticação</i>            | Autenticação JWT Bearer                                                                                          |
| <i>Hashing de Passwords</i>    | BCrypt                                                                                                           |
| <i>Base de Dados</i>           | PostgreSQL para dados da aplicação                                                                               |
| <i>Base de Dados de Testes</i> | Base de dados H2 em memória para testes automatizados isolados                                                   |
| <i>ORM</i>                     | Spring Data JPA / Hibernate                                                                                      |
| <i>Frontend</i>                | HTML, CSS e JavaScript estáticos                                                                                 |
| <i>Ferramenta de Build</i>     | Maven                                                                                                            |
| <i>Framework de Segurança</i>  | Spring Security                                                                                                  |
| <i>Testes</i>                  | JUnit 5, Spring Boot Test, Spring Security Test e MockMvc                                                        |
| <i>Testes de Segurança</i>     | Testes RBAC, testes de propriedade de casos por analistas, validação de uploads, testes de auditoria e segurança |
| <i>Integridade de Backups</i>  | Verificação SHA-256                                                                                              |

### 3. Autenticação e Validação JWT

O GhostReport utiliza autenticação JWT Bearer stateless para proteger os endpoints internos da plataforma. Os utilizadores autenticam-se através do endpoint POST /auth/login, enviando username e password. Quando as credenciais são válidas, a aplicação devolve um token JWT contendo a identidade do utilizador e o respetivo papel.

Após autenticação com sucesso, os pedidos para endpoints protegidos devem incluir o token no header HTTP:

Authorization: Bearer <token>

O fluxo de autenticação funciona da seguinte forma:

1. O utilizador envia username e password para POST /auth/login;
2. O AuthController encaminha o pedido para o AuthService;
3. O AuthService utiliza o AuthenticationManager do Spring Security para validar as credenciais;
4. A password é verificada através de BCryptPasswordEncoder;
5. O JwtService gera um token JWT assinado;
6. O JwtAuthenticationFilter valida o token nos pedidos protegidos;
7. O Spring Security aplica as regras de autorização definidas em SecurityConfig.

#### Localizações Relevantes

| <b>Ficheiro</b>                      | <b>Responsabilidade</b>                                      |
|--------------------------------------|--------------------------------------------------------------|
| <i>AuthController.java</i>           | Expõe o endpoint de login                                    |
| <i>AuthService.java</i>              | Valida credenciais e devolve a resposta de autenticação      |
| <i>JwtService.java</i>               | Gera, assina e valida tokens JWT                             |
| <i>JwtAuthenticationFilter.java</i>  | Intercepta pedidos e autentica utilizadores com Bearer token |
| <i>CustomUserDetailsService.java</i> | Carrega utilizadores a partir da base de dados               |
| <i>SecurityConfig.java</i>           | Define autenticação stateless, filtros e regras RBAC         |

#### 3.1 Estrutura do Token JWT

O JwtService gera tokens JWT utilizando o algoritmo HS256, baseado em HMAC-SHA256.

O token inclui os seguintes claims principais:

| <b>Claim</b> | <b>Descrição</b>                                |
|--------------|-------------------------------------------------|
| <i>sub</i>   | Username do utilizador autenticado              |
| <i>role</i>  | Papel do utilizador (ADMIN, ANALYST ou AUDITOR) |
| <i>iat</i>   | Timestamp de emissão                            |
| <i>exp</i>   | Timestamp de expiração                          |

O segredo utilizado para assinatura do token é configurado através da propriedade:  
`ghostreport.jwt.secret`

Em ambiente normal, esta propriedade é fornecida através da variável de ambiente:  
`JWT_SECRET`

O serviço exige que o segredo tenha pelo menos 32 caracteres, evitando utilização de chaves demasiado fracas.

A expiração do token é configurável através da propriedade:  
`ghostreport.jwt.expiration-seconds`

Por omissão, o valor utilizado é 3600 segundos.

## 3.2 Endpoint de Autenticação

| <b>Endpoint</b>               | <b>Acesso</b> | <b>Objetivo</b>                      |
|-------------------------------|---------------|--------------------------------------|
| <code>POST /auth/login</code> | Público       | Autenticar utilizador e devolver JWT |

```
alex@MacBook-Pro-de-Alex ~ % curl -i -X POST http://localhost:8081/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"Admin123!"}'
HTTP/1.1 200
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; object-src 'none'; frame-ancestors 'none'
Referrer-Policy: no-referrer
Content-Type: application/json
Transfer-Encoding: chunked
Date: Fri, 15 May 2020 20:16:40 GMT

{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbGlzInJvbnUiOiJBRE1JTlIsImhhbmciOiMTc3ODg3NjIwMCwiZXhwIjozNDc5ODAwfQ.zPqt1278zfqEik0Zha9evTeg648w1RZiQc81_NK0sfU",
"tokenType":"Bearer","username":"admin","role":"ADMIN","expiresInSeconds":3600}
alex@MacBook-Pro-de-Alex ~ %
```

Figura 1 - Exemplo de resposta de autenticação

## 3.3 Configuração Stateless

A aplicação encontra-se configurada em modo stateless:

```
.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

Isto significa que o backend não mantém sessões autenticadas no servidor. Cada pedido protegido deve incluir um Bearer token válido.



A proteção CSRF encontra-se desativada porque os endpoints protegidos utilizam autenticação baseada em Bearer tokens stateless e não sessões server-side baseadas em cookies:

```
.csrf(csrf -> csrf.disable())
```

A autenticação Basic Authentication também foi desativada:

```
.httpBasic(httpBasic -> httpBasic.disable())
```

Desta forma, os endpoints protegidos utilizam exclusivamente o fluxo JWT.

### 3.4 Endpoints Públicos e Protegidos

O GhostReport separa endpoints públicos, utilizados no fluxo anónimo de denúncias, dos endpoints autenticados da área de backoffice.

#### Endpoints Públicos

| <i>Endpoint</i>                       | <i>Objetivo</i>                       |
|---------------------------------------|---------------------------------------|
| <i>GET /</i>                          | Página pública inicial                |
| <i>GET /index.html</i>                | Interface pública                     |
| <i>GET /submit.html</i>               | Página de submissão anónima           |
| <i>GET /track.html</i>                | Página de acompanhamento de denúncias |
| <i>GET /analyst.html</i>              | Interface frontend do analista        |
| <i>GET /admin.html</i>                | Interface frontend do administrador   |
| <i>GET /auditor.html</i>              | Interface frontend do auditor         |
| <i>GET /js/**</i>                     | Ficheiros JavaScript estáticos        |
| <i>POST /auth/login</i>               | Login de utilizadores                 |
| <i>POST /reports</i>                  | Criação de denúncia anónima           |
| <i>POST /reports/verify</i>           | Verificação de tracking code          |
| <i>POST /reports/{id}/attachments</i> | Upload anónimo de evidências          |

As páginas HTML do backoffice são públicas enquanto ficheiros estáticos. No entanto, os dados e operações utilizados por essas páginas continuam protegidos através dos endpoints `/admin/**`, `/analyst/**` e `/audit/**`.

## Endpoints Protegidos

| <b>Grupo de Endpoints</b>            | <b>Papel Necessário</b> |
|--------------------------------------|-------------------------|
| <i>/admin/**</i>                     | ADMIN                   |
| <i>/analyst/**</i>                   | ANALYST ou ADMIN        |
| <i>/audit/**</i>                     | AUDITOR ou ADMIN        |
| <i>Outros endpoints não públicos</i> | Utilizador autenticado  |

### 3.5 Propriedades de Segurança da Autenticação

| <b>Propriedade de Segurança</b>            | <b>Implementação</b>                |
|--------------------------------------------|-------------------------------------|
| <i>Autenticação stateless</i>              | JWT Bearer tokens                   |
| <i>Assinatura do token</i>                 | HMAC-SHA256 (HS256)                 |
| <i>Claims mínimas</i>                      | sub, role, iat, exp                 |
| <i>Segredo externo</i>                     | JWT_SECRET / ghostreport.jwt.secret |
| <i>Expiração configurável</i>              | ghostreport.jwt.expiration-seconds  |
| <i>Proteção de passwords</i>               | BCryptPasswordEncoder               |
| <i>Filtro centralizado de autenticação</i> | JwtAuthenticationFilter             |
| <i>Autorização baseada em papéis</i>       | Regras em SecurityConfig            |
| <i>Acessos sem token válido</i>            | 401 Unauthorized                    |
| <i>Acessos autenticados sem permissão</i>  | 403 Forbidden                       |
| <i>Separação de funções</i>                | Papéis ADMIN, ANALYST e AUDITOR     |

## 4. Controlo de Acessos Baseado em Papéis (RBAC)

O controlo de acessos do GhostReport é implementado através de Spring Security e regras RBAC aplicadas centralmente em SecurityConfig.java.

A aplicação utiliza autenticação stateless baseada em JWT. Depois de o JwtAuthenticationFilter validar o Bearer token, o Spring Security aplica regras de autorização com base no papel do utilizador autenticado.

Para além da proteção ao nível dos endpoints, existem verificações adicionais na lógica de negócio para controlo de propriedade de casos, acesso a anexos e geração de pacotes de evidência. Esta combinação permite aplicar defesa em profundidade: o endpoint é protegido pela role do utilizador e os serviços confirmam adicionalmente se existe autorização sobre o recurso concreto.

## 4.1 Papéis

| <b>Papel</b>               | <b>Permissões</b>                                                                                                                                                                       |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Denunciante Anónimo</i> | Criar denúncias, verificar denúncias através do tracking code e enviar anexos associados à denúncia mediante código válido                                                              |
| <i>ANALYST</i>             | Visualizar denúncias não atribuídas e denúncias atribuídas ao próprio, assumir casos, atualizar estado, prioridade e notas, descarregar anexos autorizados e gerar pacotes de evidência |
| <i>AUDITOR</i>             | Consultar logs de auditoria, alertas de segurança, histórico de casos fechados, verificar pacotes de evidência e consultar/verificar metadados de backups                               |
| <i>ADMIN</i>               | Criar e listar utilizadores, consultar informação administrativa e de auditoria, executar operações de backup administrativo e aceder também às funcionalidades de analista             |

## 4.2 Aplicação das Regras RBAC

Em vez de utilizar um componente personalizado do tipo RoleGuard, o GhostReport aplica o controlo RBAC principalmente através de regras definidas no Spring Security:

```
.requestMatchers("/admin/**").hasRole("ADMIN")
```

```
.requestMatchers("/analyst/**").hasAnyRole("ANALYST", "ADMIN")
```

```
.requestMatchers("/audit/**").hasAnyRole("AUDITOR", "ADMIN")
```

Esta abordagem permite centralizar a autorização ao nível dos endpoints no SecurityConfig, enquanto permissões específicas do domínio são verificadas nos serviços, como ReportService, CaseReviewService, CasePackageService e BackupService.

## 4.3 Matriz de Acesso dos Controladores

| <b>Controlador</b>       | <b>Endpoint</b>                                  | <b>Papel Necessário</b>                             |
|--------------------------|--------------------------------------------------|-----------------------------------------------------|
| <i>AuthController</i>    | POST /auth/login                                 | Público                                             |
| <i>ReportController</i>  | POST /reports                                    | Público                                             |
| <i>ReportController</i>  | POST /reports/verify                             | Público + tracking code válido                      |
| <i>ReportController</i>  | POST /reports/{id}/attachments                   | Público + tracking code válido/contexto da denúncia |
| <i>AnalystController</i> | GET /analyst/reports                             | ANALYST ou ADMIN                                    |
| <i>AnalystController</i> | GET /analyst/my-cases                            | ANALYST ou ADMIN                                    |
| <i>AnalystController</i> | GET /analyst/reports/{id}/case-review            | ANALYST ou ADMIN                                    |
| <i>AnalystController</i> | GET /analyst/reports/{id}/attachments            | ANALYST ou ADMIN                                    |
| <i>AnalystController</i> | GET /analyst/attachments/{attachmentId}/download | ANALYST ou ADMIN                                    |

|                              |                                                     |                  |
|------------------------------|-----------------------------------------------------|------------------|
| <i>AnalystController</i>     | POST /analyst/reports/{id}/assign                   | ANALYST ou ADMIN |
| <i>AnalystController</i>     | PATCH /analyst/reports/{id}/status                  | ANALYST ou ADMIN |
| <i>AnalystController</i>     | PATCH /analyst/reports/{id}/priority                | ANALYST ou ADMIN |
| <i>AnalystController</i>     | PATCH /analyst/reports/{id}/notes                   | ANALYST ou ADMIN |
| <i>AnalystController</i>     | POST /analyst/reports/{id}/case-package             | ANALYST ou ADMIN |
| <i>AuditController</i>       | GET /audit/logs                                     | AUDITOR ou ADMIN |
| <i>AuditController</i>       | GET /audit/security-alerts                          | AUDITOR ou ADMIN |
| <i>AuditController</i>       | GET /audit/cases/closed                             | AUDITOR ou ADMIN |
| <i>AuditController</i>       | GET /audit/cases/{reportId}/evidence-package/verify | AUDITOR ou ADMIN |
| <i>AuditController</i>       | GET /audit/backups                                  | AUDITOR ou ADMIN |
| <i>AuditController</i>       | GET /audit/backups/{filename}/verify                | AUDITOR ou ADMIN |
| <i>AuditController</i>       | GET /audit/backups/{filename}/manifest              | AUDITOR ou ADMIN |
| <i>AdminController</i>       | GET /admin/users                                    | ADMIN            |
| <i>AdminController</i>       | POST /admin/users                                   | ADMIN            |
| <i>AdminController</i>       | GET /admin/audit-logs                               | ADMIN            |
| <i>AdminController</i>       | GET /admin/security-alerts                          | ADMIN            |
| <i>AdminBackupController</i> | POST /admin/backups                                 | ADMIN            |
| <i>AdminBackupController</i> | GET /admin/backups                                  | ADMIN            |
| <i>AdminBackupController</i> | GET /admin/backups/{filename}/download              | ADMIN            |
| <i>AdminBackupController</i> | POST /admin/backups/{filename}/verify               | ADMIN            |
| <i>AdminBackupController</i> | POST /admin/backups/{filename}/restore              | ADMIN            |

#### 4.4 Regra de Propriedade dos Casos

As verificações de ownership encontram-se distribuídas pelos seguintes componentes:

| <b>Ficheiro</b>                | <b>Responsabilidade</b>                                        |
|--------------------------------|----------------------------------------------------------------|
| <i>ReportService.java</i>      | Filtra casos visíveis e valida acesso a anexos                 |
| <i>CaseReviewService.java</i>  | Controla atribuição, prioridade, notas e ownership             |
| <i>CasePackageService.java</i> | Garante que analistas apenas geram pacotes de casos atribuídos |
| <i>ReportRepository.java</i>   | Implementa consultas de casos visíveis ao analista             |

Quando um analista assume um caso, esse caso passa a ficar associado ao respetivo analista. A partir desse momento, deixa de aparecer na fila geral para os restantes analistas. Adicionalmente, outros analistas ficam impedidos de assumir ou manipular casos já atribuídos a terceiros.

Este mecanismo reduz o risco de acessos indevidos entre analistas e reforça o princípio do menor privilégio.

## 5. DTOs e Proteção Contra Mass Assignment

Os DTOs são utilizados para separar a superfície da API das entidades persistentes da aplicação. Esta abordagem reduz o risco de mass assignment e impede exposição indevida de campos internos das entidades.

Entre os principais DTOs utilizados encontram-se:

| <b>DTO</b>                                 | <b>Objetivo</b>                                                      |
|--------------------------------------------|----------------------------------------------------------------------|
| <i>LoginRequest</i>                        | Pedido de autenticação com username e password                       |
| <i>AuthResponse</i>                        | Resposta de autenticação JWT                                         |
| <i>CreateReportRequest</i>                 | Pedido de criação de denúncia anónima                                |
| <i>CreateReportResponse</i>                | Resultado da criação da denúncia, incluindo código de acompanhamento |
| <i>VerifyTrackingCodeRequest</i>           | Pedido de verificação de código de acompanhamento                    |
| <i>ReportResponse</i>                      | Representação segura de uma denúncia                                 |
| <i>AttachmentResponse</i>                  | Metadados de anexos enviados                                         |
| <i>AttachmentListResponse</i>              | Representação da lista de anexos                                     |
| <i>DownloadRequest</i>                     | Pedido seguro de download de anexos                                  |
| <i>CreateUserRequest</i>                   | Pedido de criação de utilizador administrativo                       |
| <i>UserResponse</i>                        | Representação segura de utilizadores                                 |
| <i>UpdateReportStatusRequest</i>           | Pedido de atualização de estado                                      |
| <i>UpdatePriorityRequest</i>               | Pedido de atualização de prioridade                                  |
| <i>UpdateNotesRequest</i>                  | Pedido de atualização de notas                                       |
| <i>CaseReviewResponse</i>                  | Estado do caso devolvido ao analista                                 |
| <i>CasePackageResponse</i>                 | Resultado da geração de pacote de evidências                         |
| <i>BackupVerificationResponse</i>          | Resultado da verificação de integridade de backups                   |
| <i>EvidencePackageVerificationResponse</i> | Resultado da verificação de integridade de evidências                |

## 6. Validação e Sanitização de Entradas

A segurança das entradas no GhostReport é implementada através de múltiplas camadas de validação. Estas camadas reduzem o risco de processamento de dados inválidos, abuso dos endpoints públicos, manipulação de caminhos e armazenamento inseguro de ficheiros enviados por utilizadores.

A validação ocorre em diferentes níveis da aplicação:

| <b>Camada</b>                     | <b>Implementação</b>                                                       |
|-----------------------------------|----------------------------------------------------------------------------|
| <i>DTOs</i>                       | Jakarta Bean Validation                                                    |
| <i>Controllers</i>                | @Valid, @RequestBody, @RequestParam e validação de parâmetros obrigatórios |
| <i>Domínio</i>                    | Primitivas como SafeFilename e TrackingCode                                |
| <i>Serviços</i>                   | Validações de negócio, autorização e integridade                           |
| <i>Armazenamento de ficheiros</i> | Normalização de paths, MIME type, extensão e magic bytes                   |

Esta abordagem aplica defesa em profundidade sobre os principais pontos de entrada da aplicação.

### 6.1 Bean Validation

A primeira camada de validação é aplicada nos DTOs localizados em:

src/main/java/com/ghostreport/dto/

Os DTOs definem a superfície de entrada da API e evitam exposição direta das entidades persistentes aos pedidos externos.

A aplicação utiliza Jakarta Bean Validation através de anotações como:

| <b>Anotação</b> | <b>Objetivo</b>                                              |
|-----------------|--------------------------------------------------------------|
| @NotBlank       | Impedir campos obrigatórios vazios                           |
| @Email          | Validar formato de email                                     |
| @Size           | Restringir tamanho mínimo ou máximo                          |
| @Pattern        | Validar formatos específicos                                 |
| @Valid          | Acionar validação automática do DTO recebido pelo controller |

As principais validações implementadas incluem:

| <b>DTO</b>                       | <b>Campo</b> | <b>Validação</b>                                                            |
|----------------------------------|--------------|-----------------------------------------------------------------------------|
| <i>CreateReportRequest</i>       | title        | Obrigatório, máximo 200 caracteres                                          |
| <i>CreateReportRequest</i>       | description  | Obrigatório, máximo 4000 caracteres                                         |
| <i>CreateReportRequest</i>       | category     | Obrigatório, máximo 100 caracteres                                          |
| <i>LoginRequest</i>              | username     | Obrigatório                                                                 |
| <i>LoginRequest</i>              | password     | Obrigatório                                                                 |
| <i>CreateUserRequest</i>         | username     | Obrigatório                                                                 |
| <i>CreateUserRequest</i>         | email        | Obrigatório e formato de email                                              |
| <i>CreateUserRequest</i>         | password     | Entre 12 e 128 caracteres, incluindo maiúscula, minúscula, número e símbolo |
| <i>CreateUserRequest</i>         | role         | Obrigatório                                                                 |
| <i>UpdateReportStatusRequest</i> | status       | Obrigatório                                                                 |
| <i>UpdatePriorityRequest</i>     | priority     | Obrigatório                                                                 |
| <i>UpdateNotesRequest</i>        | notes        | Obrigatório, máximo 4000 caracteres                                         |
| <i>VerifyTrackingCodeRequest</i> | trackingCode | Obrigatório                                                                 |

## 6.2 Validação de Primitivas de Domínio

Para além da validação aplicada aos DTOs, o GhostReport utiliza primitivas de domínio para encapsular regras de segurança específicas.

| <b>Classe</b>                 | <b>Responsabilidade</b>                                |
|-------------------------------|--------------------------------------------------------|
| <i>SafeFilename.java</i>      | Validar nomes de ficheiro e bloquear padrões perigosos |
| <i>TrackingCode.java</i>      | Gerar e validar tracking codes                         |
| <i>ReportDescription.java</i> | Validar descrições de denúncias                        |

A classe *SafeFilename* rejeita:

- nomes vazios;
- nomes contendo ..;
- separadores / ou \;
- extensões executáveis perigosas;
- nomes demasiado longos.

A classe `TrackingCode` valida o formato:

GR-...

e utiliza `SecureRandom` para gerar códigos com elevada entropia.

Esta abordagem evita dispersar regras críticas por múltiplos controllers e serviços.

### 6.3 Proteção Contra Path Traversal e Uploads Maliciosos

A funcionalidade de upload foi desenvolvida com foco na proteção do sistema de ficheiros. Os anexos enviados são armazenados numa diretoria controlada pela aplicação, configurada através de:

`app.upload-dir`

#### Componentes Relevantes

| <i><b>Ficheiro</b></i>                      | <i><b>Responsabilidade</b></i>                              |
|---------------------------------------------|-------------------------------------------------------------|
| <i><code>FileStorageService.java</code></i> | Armazenamento, validação e leitura segura de ficheiros      |
| <i><code>SafeFilename.java</code></i>       | Validação de nomes de ficheiro                              |
| <i><code>ReportController.java</code></i>   | Endpoint público de upload                                  |
| <i><code>ReportService.java</code></i>      | Validação de tracking code e associação do anexo à denúncia |

Antes de qualquer operação de leitura ou escrita, o sistema normaliza os caminhos e confirma que permanecem dentro da diretoria base autorizada.

#### Medidas Implementadas

| <i><b>Medida</b></i>                | <i><b>Implementação</b></i>                                                              |
|-------------------------------------|------------------------------------------------------------------------------------------|
| <i>Limite de tamanho</i>            | Máximo de 10 MB por ficheiro                                                             |
| <i>MIME type whitelist</i>          | Apenas tipos explicitamente permitidos                                                   |
| <i>Validação de extensão</i>        | Extensão coerente com o MIME type                                                        |
| <i>Validação de magic bytes</i>     | Verificação da assinatura inicial do ficheiro                                            |
| <i>Sanitização do nome original</i> | Caracteres inseguros substituídos por <code>_</code>                                     |
| <i>Rejeição de path traversal</i>   | Bloqueio de <code>..</code> , <code>/</code> e <code>\</code>                            |
| <i>Bloqueio de executáveis</i>      | Rejeição de <code>.exe</code> , <code>.bat</code> , <code>.cmd</code> e <code>.sh</code> |
| <i>Nome interno aleatório</i>       | Armazenamento com UUID                                                                   |
| <i>Path normalization</i>           | Utilização de <code>toAbsolutePath().normalize()</code>                                  |



|                               |                                                                         |
|-------------------------------|-------------------------------------------------------------------------|
| Verificação de diretoria base | Confirmação de que o path resolvido permanece dentro de baseStoragePath |
|-------------------------------|-------------------------------------------------------------------------|

## 6.4 Tipos de Ficheiro Permitidos

Atualmente, a aplicação aceita apenas os seguintes tipos de ficheiro:

| <i><b>MIME Type</b></i>                                                        | <i><b>Extensão Esperada</b></i> |
|--------------------------------------------------------------------------------|---------------------------------|
| <i>application/pdf</i>                                                         | .pdf                            |
| <i>image/png</i>                                                               | .png                            |
| <i>image/jpeg</i>                                                              | .jpg ou .jpeg                   |
| <i>text/plain</i>                                                              | .txt                            |
| <i>application/vnd.openxmlformats-officedocument.wordprocessingml.document</i> | .docx                           |

A validação de magic bytes permite detetar casos em que o ficheiro declara um MIME type permitido, mas o conteúdo real não corresponde ao formato esperado.

Por exemplo:

- ficheiros PDF devem iniciar com %PDF;
- imagens PNG devem conter a assinatura PNG correta;
- ficheiros JPEG devem possuir cabeçalhos válidos.

Esta validação reduz riscos de MIME spoofing e uploads disfarçados.

## 6.5 Limitações da Validação de Upload

As validações implementadas reduzem significativamente o risco de uploads inseguros, mas não substituem soluções completas de análise antimalware.

Na implementação atual:

| <i><b>Controlado</b></i>           | <i><b>Não Implementado</b></i>          |
|------------------------------------|-----------------------------------------|
| <i>Tamanho máximo por ficheiro</i> | Malware scanning real                   |
| <i>MIME types permitidos</i>       | Sandbox de ficheiros                    |
| <i>Extensões permitidas</i>        | Quotas globais de armazenamento         |
| <i>Magic bytes</i>                 | CDR (Content Disarm and Reconstruction) |
| <i>Path traversal</i>              | Antivírus externo                       |

Assim, a implementação atual deve ser descrita como:

- validação forte de tipo;

- validação de extensão;
- validação de assinatura;
- e proteção de caminhos;

mas não como deteção real de malware.

## 6.6 Impacto de Segurança

As medidas implementadas mitigam múltiplas ameaças identificadas durante o threat modeling da aplicação.

| <b>Ameaça</b>                  | <b>Mitigação</b>                     |
|--------------------------------|--------------------------------------|
| <i>Path traversal</i>          | Normalização e validação de caminhos |
| <i>Upload de executáveis</i>   | Bloqueio de extensões perigosas      |
| <i>MIME spoofing</i>           | Validação de extensão e magic bytes  |
| <i>Sobrescrita previsível</i>  | Utilização de UUID como nome interno |
| <i>Enumeração de ficheiros</i> | Nomes internos aleatórios            |
| <i>Abuso de armazenamento</i>  | Limite máximo de 10 MB               |
| <i>Manipulação de nomes</i>    | Sanitização e SafeFilename           |

A utilização combinada de Bean Validation, primitivas de domínio e validação segura de ficheiros cria uma defesa em profundidade para os principais fluxos públicos da aplicação.

## 7. Tracking Code, Anonimato e Proteção Contra Abuso

O GhostReport permite a submissão anónima de denúncias sem necessidade de criação de conta. Este desenho protege o anonimato do denunciante, evitando a associação direta entre uma identidade autenticada e a denúncia submetida.

Após a criação de uma denúncia, o sistema gera um tracking code que permite ao denunciante acompanhar o estado do processo. Este código funciona como mecanismo de acesso ao acompanhamento da denúncia, pelo que deve ser difícil de adivinhar, não sequencial e protegido contra enumeração.

O formato utilizado é:

*GR-<token aleatório URL-safe>*

O token é gerado com SecureRandom e codificação Base64 URL-safe, reduzindo a probabilidade de geração previsível ou adivinhação de códigos válidos.

Para reforçar a confidencialidade, o tracking code não é armazenado em claro. O sistema guarda apenas o hash do código, reduzindo o impacto de uma eventual exposição da base de dados. Durante a verificação, o código fornecido pelo utilizador é comparado com o valor armazenado de forma indireta.

## 7.1 Proteção Contra Enumeração

Uma das ameaças identificadas na Phase 1 foi a enumeração de códigos de acompanhamento. Um atacante poderia tentar gerar ou testar múltiplos códigos até encontrar uma denúncia válida.

Para mitigar este risco, o GhostReport combina várias medidas:

| <b>Risco</b>                            | <b>Mitigação</b>                        |
|-----------------------------------------|-----------------------------------------|
| <i>Códigos previsíveis</i>              | Geração aleatória com SecureRandom      |
| <i>Enumeração sequencial</i>            | Formato não sequencial e token URL-safe |
| <i>Exposição em base de dados</i>       | Armazenamento do hash do tracking code  |
| <i>Respostas demasiado informativas</i> | Erros genéricos para códigos inválidos  |
| <i>Tentativas repetidas</i>             | Rate limiting nos fluxos públicos       |

O projeto inclui testes automatizados relacionados com tracking codes e enumeração, nomeadamente `TrackingCodeTest` e `TrackingCodeEnumerationTest`, reforçando os requisitos de anonimato e confidencialidade definidos na Phase 1.

## 7.2 Rate Limiting nos Fluxos Públicos

O GhostReport implementa mecanismos de rate limiting nos fluxos públicos da aplicação, com o objetivo de reduzir abuso automatizado, brute force e enumeração de recursos sensíveis.

Componentes relevantes:

| <b>Ficheiro</b>                 | <b>Responsabilidade</b>                                                    |
|---------------------------------|----------------------------------------------------------------------------|
| <i>RateLimiterService.java</i>  | Controla limites de pedidos por chave e janela temporal                    |
| <i>RateLimitProperties.java</i> | Mapeia a configuração definida nos ficheiros <code>application.yaml</code> |
| <i>ReportController.java</i>    | Aplica rate limiting nos endpoints públicos relevantes                     |

As operações atualmente protegidas incluem:

- verificação de códigos de acompanhamento;
- upload de anexos;

- download de ficheiros associados às denúncias, quando aplicável.

A implementação utiliza contadores em memória associados a uma chave identificadora da operação. Cada contador é avaliado dentro de uma janela temporal configurável. O limite atualmente configurado corresponde a 10 tentativas por chave.

Quando o limite é excedido, a API rejeita automaticamente novos pedidos com:

## HTTP 429 Too Many Requests

### 7.3 Impacto de Segurança

A combinação de tracking codes aleatórios, armazenamento por hash, respostas genéricas e rate limiting reduz significativamente o risco de enumeração de denúncias e acesso indevido a informação sensível.

Estas medidas contribuem para os seguintes objetivos de segurança definidos na Phase 1:

| <b>Objetivo</b>                                      | <b>Contribuição da implementação</b>                     |
|------------------------------------------------------|----------------------------------------------------------|
| <i>Anonimato do denunciante</i>                      | A denúncia pode ser submetida sem conta autenticada      |
| <i>Confidencialidade</i>                             | O acesso ao acompanhamento exige tracking code válido    |
| <i>Proteção contra enumeração</i>                    | Códigos aleatórios, erro genérico e rate limiting        |
| <i>Redução de impacto em caso de exposição da BD</i> | Tracking code armazenado como hash                       |
| <i>Auditabilidade</i>                                | Tentativas suspeitas podem originar alertas de segurança |

### 7.4 Limitações Atuais

Apesar de a implementação atual cumprir os objetivos da Sprint 1, existem limitações relevantes.

O rate limiting é mantido em memória, sendo adequado para ambiente de desenvolvimento, testes ou execução single-instance. Em ambientes distribuídos, múltiplas instâncias da aplicação não partilhariam o mesmo contador de tentativas.

Como melhoria futura, o sistema deverá migrar para uma solução baseada em armazenamento distribuído, como Redis, permitindo sincronização entre instâncias, maior persistência e melhor controlo de abuso em ambientes de produção.

Além disso, o sistema ainda não implementa mecanismos avançados como deteção comportamental, CAPTCHA adaptativo ou análise de reputação de origem. Esses

mecanismos poderiam ser considerados em fases futuras caso o fluxo público fosse exposto em ambiente real.

## 8. Error Handling e Prevenção de Information Disclosure

### Localização:

`ghostreport/src/main/java/com/ghostreport/exception/GlobalExceptionHandler.java`

O GhostReport centraliza o tratamento de erros através de um `@RestControllerAdvice`, garantindo que exceções lançadas pela API são convertidas em respostas JSON consistentes e controladas.

Esta abordagem evita exposição direta de stack traces, nomes de classes internas, paths do filesystem ou detalhes técnicos sensíveis aos clientes da aplicação.

O principal objetivo deste mecanismo é reduzir riscos de **Information Disclosure**, fornecendo mensagens suficientemente claras para o frontend e utilizadores legítimos, mas sem revelar informação interna da implementação.

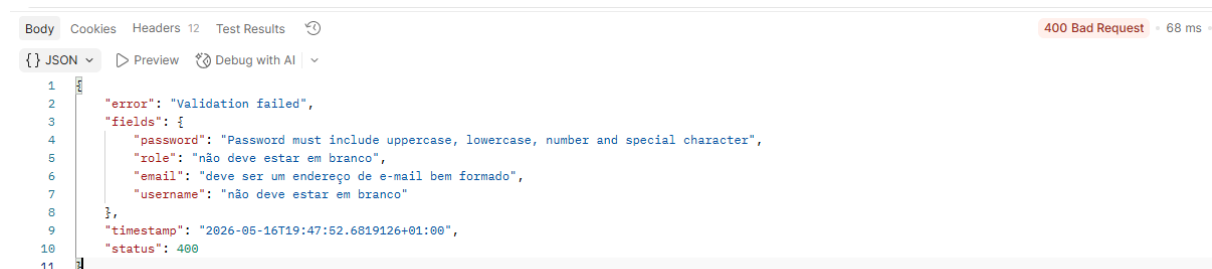
### 8.1 Estrutura das Respostas de Erro

As respostas de erro seguem uma estrutura JSON previsível e consistente.

Exemplo de erro genérico:

```
{  
  
  "timestamp": "2026-05-15T12:00:00+01:00",  
  
  "status": 400,  
  
  "error": "Validation failed"  
}
```

Nos casos de erro de validação, a resposta pode incluir detalhes dos campos inválidos:



Este formato permite ao frontend apresentar mensagens claras sem depender de páginas HTML de erro geradas automaticamente pelo Spring Boot.

## 8.2 Mapeamento de Exceções

O `GlobalExceptionHandler` converte diferentes tipos de exceção em respostas HTTP controladas.

| <b>Exceção</b>                                 | <b>HTTP Status</b>           | <b>Resposta Exposta</b>                             |
|------------------------------------------------|------------------------------|-----------------------------------------------------|
| <i>ResponseStatusException</i>                 | Status definido pela exceção | Mensagem controlada definida no serviço/controlador |
| <i>MethodArgumentNotValidException</i>         | 400                          | Validation failed + erros por campo                 |
| <i>MissingServletRequestParameterException</i> | 400                          | Missing required request parameter                  |
| <i>AccessDeniedException</i>                   | 403                          | Access denied                                       |
| <i>Exception</i>                               | 500                          | Unexpected internal error                           |

A utilização de `ResponseStatusException` permite devolver erros controlados para cenários esperados, como:

- tracking codes inválidos;
- uploads inválidos;
- ficheiros demasiado grandes;
- acessos proibidos;
- recursos inexistentes;
- conflitos de estado de casos;
- falhas de integridade em backups ou pacotes de evidência.

## 8.3 Prevenção de Information Disclosure

O tratamento global de erros reduz exposição indevida de informação através de várias medidas de proteção.

| <b>Risco</b>                                         | <b>Mitigação Implementada</b>                       |
|------------------------------------------------------|-----------------------------------------------------|
| <i>Exposição de stack traces</i>                     | <code>server.error.include-stacktrace=never</code>  |
| <i>Exposição de paths internos</i>                   | Mensagens controladas nos fluxos de upload/download |
| <i>Exposição de detalhes internos do Spring/Java</i> | Catch-all devolve Unexpected internal error         |
| <i>Exposição de campos sensíveis</i>                 | DTOs e responses controlam os dados devolvidos      |

*Validação inconsistente*

Bean Validation convertida para respostas JSON estruturadas

*Mensagens excessivamente detalhadas*

Respostas genéricas para erros de autorização

As propriedades de erro encontram-se configuradas nos ficheiros:

- *application.yaml*
- *application-dev.yaml*
- *application-test.yaml*

## 8.4 Integração com o Modelo de Segurança

O tratamento de erros encontra-se alinhado com os mecanismos de autenticação e autorização implementados no GhostReport.

Desta forma:

- pedidos sem autenticação válida recebem 401 Unauthorized;
- utilizadores autenticados sem permissões suficientes recebem 403 Forbidden;
- endpoints protegidos continuam sujeitos às regras RBAC definidas no SecurityConfig;
- acessos indevidos a funcionalidades sensíveis podem gerar security alerts;
- erros de validação são devolvidos como 400 Bad Request.

Esta separação é importante porque impede que atacantes obtenham detalhes internos da aplicação apenas através da análise das mensagens de erro.

## 8.5 Exemplos de Erros Controlados

| Cenário                                         | Resposta Esperada                 |
|-------------------------------------------------|-----------------------------------|
| <i>Tracking code inválido</i>                   | 400 Bad Request ou 404 Not Found  |
| <i>Upload sem trackingCode</i>                  | 400 Bad Request                   |
| <i>Upload com tracking code inválido</i>        | 403 Forbidden                     |
| <i>Upload de ficheiro demasiado grande</i>      | 400 Bad Request                   |
| <i>Download de ficheiro inexistente</i>         | 404 Not Found                     |
| <i>Analyst a aceder a caso de outro analyst</i> | 403 Forbidden                     |
| <i>Auditor sem permissões adequadas</i>         | 401 Unauthorized ou 403 Forbidden |
| <i>Erro inesperado</i>                          | 500 Unexpected internal error     |

Em todos os casos, a aplicação evita devolver stack traces ou detalhes internos da implementação.

## 9. Auditoria e Monitorização de Segurança

Localizações relevantes no projeto:

- **service/AuditLogService.java** — responsável pelo registo centralizado de eventos de auditoria da aplicação.
- **service/SecurityMonitoringService.java** — responsável pela deteção e geração de alertas relacionados com atividades suspeitas.
- **controller/AuditController.java** — disponibiliza os endpoints protegidos para consulta de logs e alertas de segurança.

O GhostReport implementa mecanismos de auditoria e monitorização de segurança com o objetivo de garantir rastreabilidade, deteção de comportamentos suspeitos e suporte à análise forense de incidentes. Estes mecanismos permitem acompanhar operações críticas realizadas no sistema e identificar atividades potencialmente maliciosas ou anómalas.

### Logs de Auditoria

A aplicação regista eventos relevantes através de logs de auditoria estruturados, permitindo manter histórico detalhado das ações executadas no sistema.

Os logs armazenam informações como:

- ator responsável pela ação;
- tipo de operação executada;
- tipo de alvo afetado;
- identificador do recurso associado;
- detalhes adicionais sanitizados.

Antes de serem persistidos, os detalhes dos logs são sanitizados para remover quebras de linha, caracteres de controlo e conteúdo potencialmente perigoso, reduzindo o risco de ataques de log injection ou manipulação maliciosa de registos de auditoria.

Os logs de auditoria suportam:

- rastreabilidade de operações críticas;
- análise posterior de incidentes;



- verificação de ações administrativas;
- acompanhamento de operações sobre denúncias e evidências digitais.

## Alertas de Segurança

Para além dos logs tradicionais, o sistema implementa mecanismos de monitorização ativa capazes de gerar alertas automáticos perante comportamentos suspeitos ou violações de segurança detetadas pela aplicação.

Os principais alertas atualmente suportados incluem:

- **TRACKING\_CODE\_ENUMERATION** — múltiplas tentativas inválidas de tracking codes;
- **SUSPICIOUS\_UPLOAD\_ACTIVITY** — uploads repetidamente rejeitados;
- **PATH\_TRAVERSAL\_ATTEMPT** — tentativas de manipulação de caminhos em uploads;
- **BACKUP\_PATH\_TRAVERSAL\_ATTEMPT** — tentativas de path traversal relacionadas com backups;
- **BACKUP\_UNAUTHORIZED\_ATTEMPT** — acessos não autorizados a operações de backup;
- **BACKUP\_INTEGRITY\_FAILURE** — falhas de integridade detetadas durante validação de backups.

Os alertas gerados podem ser consultados por utilizadores com permissões adequadas através dos endpoints protegidos:

- GET /audit/security-alerts
- GET /admin/security-alerts

A integração destes mecanismos reforça a capacidade de deteção precoce de incidentes, aumenta a auditabilidade do sistema e contribui para uma abordagem de defesa em profundidade na arquitetura de segurança do GhostReport.

## 10. Pacotes de Evidência e Integridade de Backups

O GhostReport implementa mecanismos de preservação e verificação de evidência digital, com foco em integridade, rastreabilidade e recuperação controlada de dados. Estas funcionalidades encontram-se distribuídas entre os serviços de pacotes de evidência, backups e auditoria.

| <b>Componente</b>                     | <b>Responsabilidade</b>                                                                    |
|---------------------------------------|--------------------------------------------------------------------------------------------|
| <i>CasePackageService.java</i>        | Geração e verificação de pacotes de evidência associados a casos                           |
| <i>BackupService.java</i>             | Criação, listagem, verificação e staging de backups                                        |
| <i>AuditController.java</i>           | Endpoints de leitura e verificação acessíveis ao auditor                                   |
| <i>AdminBackupController.java</i>     | Endpoints administrativos para criação, download, verificação e restore staging de backups |
| <i>SecurityMonitoringService.java</i> | Geração de alertas perante eventos suspeitos relacionados com backups e integridade        |

### 10.1 Pacotes de Evidência

O sistema permite gerar pacotes de evidência associados a denúncias tratadas pela plataforma. Estes pacotes destinam-se a preservar os elementos relevantes de um caso e permitir a sua validação posterior.

A geração de pacotes de evidência apenas é permitida para casos encerrados, ou seja, casos com estado RESOLVED ou REJECTED. Esta restrição evita a criação de pacotes finais enquanto o caso ainda se encontra em análise ou sujeito a alterações.

Durante a geração, o sistema cria uma diretoria *case\_package* dentro da pasta do relatório e inclui:

| <b>Elemento</b>               | <b>Descrição</b>                                                                |
|-------------------------------|---------------------------------------------------------------------------------|
| <i>case_summary.txt</i>       | Resumo textual do caso, incluindo estado, categoria, descrição e notas internas |
| <i>evidence_manifest.json</i> | Manifest com os anexos incluídos e respetivos hashes                            |
| <i>integrity_hashes.txt</i>   | Lista de hashes SHA-256 dos ficheiros incluídos                                 |
| <i>attachments/</i>           | Cópia dos anexos associados à denúncia                                          |

Cada anexo incluído no pacote é copiado para a pasta do pacote e validado através de um hash SHA-256. O manifest regista o nome original, o nome armazenado e o hash esperado de cada ficheiro.

Esta abordagem permite verificar posteriormente se os ficheiros incluídos no pacote foram alterados, removidos ou substituídos.

### 10.2 Verificação de Pacotes de Evidência

O GhostReport disponibiliza um endpoint de verificação de pacotes de evidência através da área de auditoria:

GET /audit/cases/{reportId}/evidence-package/verify

Durante a verificação, o sistema:

1. Confirma que o caso existe;
2. Confirma que o caso está encerrado;
3. Lê o ficheiro evidence\_manifest.json;
4. Valida se o reportId do manifest corresponde ao caso pedido;
5. Confirma que a lista de anexos no manifest é válida;
6. Verifica se cada ficheiro existe na pasta attachments/;
7. Calcula novamente o SHA-256 de cada ficheiro;
8. Compara o hash atual com o hash registado no manifest.

Se algum ficheiro estiver em falta, se o manifest estiver inválido ou se existir divergência de hash, a verificação falha. Isto permite detetar alterações indevidas nos ficheiros após a geração do pacote.

### 10.3 Controlo de Acesso aos Pacotes

A geração de pacotes de evidência respeita regras de autorização. Administradores podem gerar pacotes, enquanto analistas apenas podem gerar pacotes de casos que lhes estejam atribuídos.

A verificação dos pacotes encontra-se disponível na área /audit/\*\*, protegida pelo perfil de auditoria. Esta separação distingue operações de análise e gestão das operações de validação e auditoria.

Esta abordagem contribui para o princípio de menor privilégio, evitando que qualquer utilizador autenticado consiga validar ou consultar evidências fora do seu âmbito.

### 10.4 Backups da Aplicação

O GhostReport inclui funcionalidades de backup em formato ZIP. Os backups exportam dados relevantes da aplicação e ficheiros carregados, permitindo recuperação controlada e análise posterior.

Os backups incluem exportações JSON das principais entidades:

| <b>Exportação</b>              | <b>Conteúdo</b>                     |
|--------------------------------|-------------------------------------|
| <i>db/reports.json</i>         | Denúncias                           |
| <i>db/attachments.json</i>     | Metadados dos anexos                |
| <i>db/case-reviews.json</i>    | Revisões de casos                   |
| <i>db/users.json</i>           | Utilizadores                        |
| <i>db/audit-logs.json</i>      | Logs de auditoria                   |
| <i>db/security-alerts.json</i> | Alertas de segurança                |
| <i>files/</i>                  | Ficheiros carregados pela aplicação |
| <i>manifest.json</i>           | Manifest do backup                  |

O nome dos backups segue um formato controlado:

*ghostreport-backup-yyyymmdd-HHmmss.zip*

Este padrão é validado antes de operações de leitura, download, verificação ou restore, reduzindo riscos de path traversal.

## 10.5 Manifest e Hashes de Integridade

Cada backup inclui um manifest.json com metadados sobre os ficheiros incluídos. Para cada entrada, o manifest regista:

| <b>Campo</b>  | <b>Objetivo</b>                           |
|---------------|-------------------------------------------|
| <i>path</i>   | Caminho interno do ficheiro dentro do ZIP |
| <i>size</i>   | Tamanho do ficheiro                       |
| <i>sha256</i> | Hash SHA-256 esperado                     |

Além do manifest interno, o sistema gera também um ficheiro sidecar com o hash SHA-256 do ZIP completo:

*ghostreport-backup-yyyymmdd-HHmmss.zip.sha256*

Desta forma, a verificação de backup cobre dois níveis:

1. Integridade de cada ficheiro dentro do ZIP;
2. Integridade global do ficheiro ZIP.

Esta implementação não torna os backups tamper-proof de forma criptográfica, porque não existe assinatura digital nem armazenamento append-only. No entanto, permite detetar alterações, corrupção ou substituição de ficheiros através da validação de hashes.

## 10.6 Verificação de Backups

O sistema disponibiliza endpoints para validação de backups:

GET /audit/backups

GET /audit/backups/{filename}/verify

GET /audit/backups/{filename}/manifest

POST /admin/backups/{filename}/verify

Durante a verificação, o sistema:

1. Valida o nome do ficheiro de backup;
2. Confirma que o backup existe;
3. Lê o manifest.json;
4. Verifica se a estrutura do manifest é válida;
5. Confirma que todos os ficheiros listados existem no ZIP;
6. Calcula novamente o SHA-256 de cada ficheiro;
7. Compara os hashes calculados com os hashes esperados;
8. Valida o total de ficheiros indicado no manifest;
9. Se existir ficheiro .sha256, valida também o hash global do ZIP.

Se a validação falhar, o sistema rejeita o backup e regista o evento como falha de integridade.

## 10.7 Restore Staging

O GhostReport inclui uma operação de restore controlado, disponível para administradores:

POST /admin/backups/{filename}/restore

Antes de extrair qualquer ficheiro, o backup é validado através do processo de integridade anteriormente descrito. Após validação, o conteúdo é extraído para uma área de staging:

backups/restore-staging/

A operação não substitui automaticamente a base de dados nem os ficheiros ativos da aplicação. O restore atual corresponde, portanto, a um restore staging: o conteúdo é validado e extraído para uma diretoria controlada, permitindo revisão manual antes de qualquer recuperação efetiva.

Esta abordagem reduz o risco de sobrescrever dados válidos com um backup corrompido ou malicioso.

## 10.8 Proteções Contra Path Traversal e ZIP Slip

As operações de backup e restore incluem validações específicas para evitar manipulação de caminhos.

| <b>Proteção</b>                  | <b>Implementação</b>                                                   |
|----------------------------------|------------------------------------------------------------------------|
| <i>Nome de backup restrito</i>   | Regex para aceitar apenas ghostreport-backup-yyyyMMdd-HHmmss.zip       |
| <i>Diretória base controlada</i> | Validação de que os caminhos resolvidos permanecem dentro de backupDir |
| <i>Entradas ZIP seguras</i>      | Rejeição de entradas com .., caminhos absolutos ou \                   |
| <i>Restore controlado</i>        | Extração apenas para restore-staging                                   |
| <i>Alertas de segurança</i>      | Registo de tentativas suspeitas                                        |

Estas validações mitigam riscos como path traversal e ZIP Slip durante operações de download, verificação e extração de backups.

## 10.9 Alertas e Auditoria

As operações críticas de backup são registadas através de audit logs.

| <b>Evento</b>                | <b>Descrição</b>             |
|------------------------------|------------------------------|
| <i>BACKUP_CREATED</i>        | Backup criado com sucesso    |
| <i>BACKUP_DOWNLOADED</i>     | Backup descarregado          |
| <i>BACKUP_VALIDATED</i>      | Backup validado com sucesso  |
| <i>BACKUP_RESTORE_STAGED</i> | Backup extraído para staging |
| <i>BACKUP_ERROR</i>          | Falha em operação de backup  |

Além dos audit logs, o sistema cria alertas de segurança para situações suspeitas.

| <b>Alerta</b>                        | <b>Situação</b>                                          |
|--------------------------------------|----------------------------------------------------------|
| <i>BACKUP_PATH_TRAVERSAL_ATTEMPT</i> | Tentativa de utilizar nome ou caminho inválido           |
| <i>BACKUP_MISSING_DOWNLOAD</i>       | Tentativa de descarregar backup inexistente              |
| <i>BACKUP_INTEGRITY_FAILURE</i>      | Falha de validação de integridade                        |
| <i>BACKUP_UNAUTHORIZED_ATTEMPT</i>   | Tentativa não autorizada de acesso a endpoints de backup |

Estes alertas reforçam a capacidade de deteção e ajudam administradores e auditores a identificar comportamentos anómalos.

## 10.10 Limitações Atuais

Apesar dos mecanismos implementados, continuam a existir limitações relevantes.

| <b>Limitação</b>                                                        | <b>Impacto</b>                                                                                 |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <i>Backups não são assinados digitalmente</i>                           | A integridade é baseada em hashes e não em prova criptográfica de autoria                      |
| <i>Backups não são append-only</i>                                      | Um utilizador com acesso ao sistema de ficheiros pode alterar ou apagar artefactos             |
| <i>Restore não reimporta automaticamente para a base de dados ativa</i> | A recuperação permanece em staging para validação manual                                       |
| <i>Não existe encriptação dos backups</i>                               | O conteúdo exportado deve ser protegido através de controlos de acesso ao sistema de ficheiros |
| <i>Não existe política de retenção avançada</i>                         | A gestão do ciclo de vida dos backups pode ser melhorada                                       |

Estas limitações devem ser consideradas em futuras iterações, sobretudo caso o sistema evolua para contexto de produção.

## 10.11 Relação com Requisitos de Segurança

As funcionalidades de pacotes de evidência e backups suportam vários requisitos definidos na Phase 1.

| <b>Requisito</b>                          | <b>Implementação</b>                        |
|-------------------------------------------|---------------------------------------------|
| <i>Preservação de evidência</i>           | Pacotes de evidência por caso               |
| <i>Integridade de ficheiros</i>           | Hashes SHA-256                              |
| <i>Auditabilidade</i>                     | Audit logs e endpoints de auditoria         |
| <i>Deteção de tampering</i>               | Verificação de manifests e hashes           |
| <i>Recuperação controlada</i>             | Backups ZIP e restore staging               |
| <i>Proteção contra path traversal</i>     | Validação de nomes, caminhos e entradas ZIP |
| <i>Monitorização de eventos suspeitos</i> | Security alerts                             |

## 11. Pipeline DevSecOps

O GhostReport integra uma pipeline DevSecOps automatizada através de GitHub Actions. Para melhorar desempenho, isolamento e rastreabilidade, a pipeline foi

dividida em workflows independentes, cada um responsável por uma dimensão específica da validação.

| <b>Workflow</b>                 | <b>Objetivo</b>                                                        |
|---------------------------------|------------------------------------------------------------------------|
| <i>ci-tests.yml</i>             | Build e execução dos testes automatizados                              |
| <i>sast-spotbugs.yml</i>        | Análise estática de código Java com SpotBugs                           |
| <i>sca-dependency-check.yml</i> | Análise de vulnerabilidades em dependências com OWASP Dependency-Check |
| <i>secret-scan-gitleaks.yml</i> | Deteção de segredos hardcoded com Gitleaks                             |
| <i>dast-zap.yml</i>             | Análise dinâmica baseline com OWASP ZAP                                |

Os workflows são executados automaticamente em eventos de push e pull\_request para as branches main e develop.

| <b>Evento</b>       | <b>Branches</b> |
|---------------------|-----------------|
| <i>push</i>         | main, develop   |
| <i>pull_request</i> | main, develop   |

Esta organização permite validar alterações de forma contínua, reduzir regressões e produzir artefactos técnicos que servem como evidência das verificações realizadas.

### 11.1 Etapas da Pipeline

A pipeline cobre diferentes níveis de validação da aplicação.

| <b>Etapa</b>                        | <b>Objetivo</b>                                              |
|-------------------------------------|--------------------------------------------------------------|
| <i>Checkout do repositório</i>      | Obter o código-fonte da versão em análise                    |
| <i>Setup Java 17</i>                | Preparar o ambiente compatível com o projeto                 |
| <i>Maven cache</i>                  | Reutilizar dependências Maven e acelerar execuções           |
| <i>PostgreSQL service container</i> | Disponibilizar uma base de dados realista para testes e DAST |
| <i>Maven build</i>                  | Validar compilação da aplicação                              |
| <i>Maven tests</i>                  | Executar testes automatizados                                |
| <i>SpotBugs</i>                     | Identificar padrões inseguros e más práticas no código Java  |
| <i>Dependency-Check</i>             | Identificar dependências com CVEs conhecidos                 |
| <i>Gitleaks</i>                     | Detetar possíveis segredos hardcoded                         |
| <i>OWASP ZAP Baseline</i>           | Executar DAST passivo sobre a aplicação em execução          |



## 11.2 Base de Dados em Ambiente CI

Os workflows que necessitam da aplicação em execução utilizam PostgreSQL 16 através de um service container, aproximando o ambiente de CI do ambiente real da aplicação.

| <i><b>Propriedade</b></i> | <i><b>Valor</b></i> |
|---------------------------|---------------------|
| <i>Imagem</i>             | postgres:16         |
| <i>Base de dados</i>      | ghostreport         |
| <i>Utilizador</i>         | postgres            |
| <i>Porta</i>              | 5432                |

As credenciais são fornecidas por variáveis de ambiente durante a execução:

*DB\_URL=jdbc:postgresql://localhost:5432/ghostreport*

*DB\_USERNAME=postgres*

*DB\_PASSWORD=user*

Esta abordagem evita hardcoding de credenciais no código-fonte e permite isolar a configuração do ambiente de CI.

## 11.3 Build e Testes Automatizados

O workflow de testes executa build e testes automatizados com Maven. Esta fase valida se o projeto compila corretamente e se os controlos funcionais e de segurança continuam a comportar-se como esperado.

As áreas cobertas pelos testes incluem:

| <i><b>Área</b></i>        | <i><b>Objetivo</b></i>              |
|---------------------------|-------------------------------------|
| <i>RBAC</i>               | Verificação de permissões por papel |
| <i>Autenticação</i>       | Proteção de endpoints autenticados  |
| <i>Ownership de casos</i> | Isolamento entre analistas          |
| <i>Uploads</i>            | Validação de ficheiros e MIME types |
| <i>Path traversal</i>     | Rejeição de caminhos maliciosos     |
| <i>Tracking codes</i>     | Resistência a enumeração            |
| <i>Audit logs</i>         | Registo de operações críticas       |

|                          |                              |
|--------------------------|------------------------------|
| <i>Security alerts</i>   | Registo de eventos suspeitos |
| <i>Backups</i>           | Verificação de integridade   |
| <i>Evidence packages</i> | Geração e validação segura   |

Desta forma, o workflow funciona como barreira automática contra regressões em funcionalidades críticas de segurança.

#### 11.4 SAST com SpotBugs

A pipeline executa análise estática de código Java com SpotBugs. O código é compilado antes da análise, garantindo que o SpotBugs avalia as classes geradas e produz relatórios XML como artefactos.

Na execução analisada, o SpotBugs identificou **38 findings**.

| <i><b>Categoria</b></i>                        | <i><b>Findings</b></i> |
|------------------------------------------------|------------------------|
| <i>Malicious Code / Exposed Representation</i> | 31                     |
| <i>Bad Practice</i>                            | 5                      |
| <i>Style / Dodgy Code</i>                      | 2                      |

A maioria dos findings está relacionada com exposição de objetos mutáveis, especialmente regras EI\_EXPOSE\_REP e EI\_EXPOSE\_REP2. Estes casos surgem sobretudo em classes de configuração, DTOs, entidades e serviços.

Os resultados não indicam exploração direta como SQL Injection, bypass de autenticação ou falha crítica de upload. Representam principalmente oportunidades de hardening, como uso de objetos imutáveis, defensive copies e revisão de casos em que dependências geridas pelo Spring são armazenadas por injeção de dependência.

#### 11.5 SCA com OWASP Dependency-Check

A pipeline executa OWASP Dependency-Check para analisar dependências Maven e identificar vulnerabilidades conhecidas.

Na execução analisada, foram avaliadas **53 dependências**. O relatório identificou vulnerabilidades em **11 artefactos**, totalizando **44 entradas CVE**.

| <i><b>Severidade</b></i> | <i><b>Findings</b></i> |
|--------------------------|------------------------|
| <i>Critical</i>          | 7                      |
| <i>High</i>              | 13                     |
| <i>Medium</i>            | 19                     |

As principais dependências assinaladas incluem:

- Spring Boot;
- Spring Framework;
- Spring Security;
- Tomcat embebido;
- PostgreSQL JDBC;
- Log4j API;
- Hibernate Validator;
- Angus Activation.

O relatório é produzido em formatos como HTML, XML, JSON e SARIF, permitindo análise manual e futura integração com ferramentas de code scanning.

Na configuração atual, o Dependency-Check funciona como mecanismo de evidência e monitorização, sem bloquear automaticamente o build com base nos CVEs encontrados. A triagem manual continua necessária para confirmar impacto real, false positives e versões corrigidas.

### 11.6 Secret Scanning com Gitleaks

A pipeline executa Gitleaks para identificar possíveis segredos hardcoded no repositório.

Na execução analisada, o relatório JSON produzido foi vazio:

```
[]
```

Este resultado indica que não foram detetados segredos no conteúdo analisado do repositório. O relatório é mantido como artefacto para evidência da execução.

### 11.7 DAST com OWASP ZAP Baseline

A pipeline executa análise dinâmica com OWASP ZAP Baseline contra a aplicação GhostReport em execução no ambiente CI.

Durante esta fase:

1. A aplicação é iniciada automaticamente.
2. O workflow aguarda até a aplicação estar disponível.
3. O ZAP executa um baseline scan passivo.

4. O relatório DAST e os logs da aplicação são publicados como artefactos.

Na execução analisada, a aplicação iniciou corretamente na porta 8081, e o ZAP gerou um relatório HTML. O scan não identificou vulnerabilidades críticas de exploração direta. Os findings observados correspondem principalmente a recomendações de hardening de headers HTTP e políticas de browser.

| <i>Nível de risco</i> | <i>Finding</i>                                  |
|-----------------------|-------------------------------------------------|
| <i>Medium</i>         | CSP missing fallback directive                  |
| <i>Medium</i>         | CSP allows script-src unsafe-inline             |
| <i>Medium</i>         | CSP allows style-src unsafe-inline              |
| <i>Low</i>            | Cross-Origin-Embedder-Policy missing or invalid |
| <i>Low</i>            | Cross-Origin-Opener-Policy missing or invalid   |
| <i>Low</i>            | Cross-Origin-Resource-Policy missing or invalid |
| <i>Low</i>            | Permissions-Policy header not set               |
| <i>Informational</i>  | Suspicious comments                             |
| <i>Informational</i>  | Non-storable content                            |

Como o ZAP Baseline é passivo e não autenticado, os resultados devem ser interpretados como uma primeira camada de validação dinâmica. Scans autenticados por perfil e análises mais profundas ficam como melhoria futura.

### 11.8 Artefactos de Segurança

Os workflows publicam artefactos no GitHub Actions, permitindo manter evidência técnica das verificações executadas.

| <i>Ferramenta</i>       | <i>Artefactos</i>                  |
|-------------------------|------------------------------------|
| <i>SpotBugs</i>         | Relatórios XML                     |
| <i>Dependency-Check</i> | HTML, XML, JSON, SARIF             |
| <i>Gitleaks</i>         | JSON                               |
| <i>OWASP ZAP</i>        | Relatório HTML e logs da aplicação |

Estes artefactos suportam análise posterior, revisão em pull requests, comparação entre execuções e documentação de findings e mitigações futuras.

### 11.9 Relação com SSDLC

A pipeline suporta diretamente os princípios do Secure Software Development Life Cycle (SSDLC), integrando segurança no fluxo normal de desenvolvimento.

| <b>Prática SSDLC</b>                 | <b>Implementação no GhostReport</b>        |
|--------------------------------------|--------------------------------------------|
| <i>Secure implementation</i>         | Build e testes automatizados               |
| <i>Security verification</i>         | Testes RBAC, ownership, uploads e backups  |
| <i>Static analysis</i>               | SpotBugs                                   |
| <i>Software composition analysis</i> | OWASP Dependency-Check                     |
| <i>Secret management validation</i>  | Gitleaks                                   |
| <i>Dynamic analysis</i>              | OWASP ZAP Baseline                         |
| <i>Evidence collection</i>           | Publicação de artefactos                   |
| <i>Regression prevention</i>         | Execução automática em push e pull request |

### 11.10 Limitações Atuais e Melhorias Futuras

Apesar da pipeline já incluir validações relevantes, existem melhorias planeadas para próximas sprints.

| <b>Limitação atual</b>                                       | <b>Melhoria futura</b>                                         |
|--------------------------------------------------------------|----------------------------------------------------------------|
| <i>SpotBugs gera findings que requerem triagem manual</i>    | Corrigir findings reais e documentar suppressions justificadas |
| <i>Dependency-Check não bloqueia o build</i>                 | Definir thresholds de severidade após triagem                  |
| <i>CVEs ainda não foram avaliados quanto ao impacto real</i> | Atualizar dependências e documentar false positives            |
| <i>ZAP Baseline é passivo e não autenticado</i>              | Criar scans autenticados por perfil                            |
| <i>ZAP cobre sobretudo endpoints públicos</i>                | Expandir cobertura para fluxos internos                        |
| <i>Headers HTTP ainda têm recomendações de hardening</i>     | Melhorar CSP e adicionar headers modernos                      |
| <i>Não existe policy gate consolidado</i>                    | Definir critérios mínimos para aprovação                       |

## 12. Threat Modeling: Ameaças Identificadas e Mitigações Implementadas

A tabela seguinte estabelece a relação entre as ameaças identificadas durante a Phase 1 e os mecanismos de mitigação implementados no Sprint 1 da Phase 2. Esta

abordagem permite demonstrar rastreabilidade entre o processo de threat modeling e os controlos técnicos efetivamente desenvolvidos no GhostReport.

| <b>Ameaça Identificada na Phase 1</b>           | <b>Categoria STRIDE</b>                                | <b>Risco Identificado</b> | <b>Mitigação Definida na Phase 1</b>                                              | <b>Implementação no Sprint 1</b>                                                                                                                                                                           | <b>Estado</b>             |
|-------------------------------------------------|--------------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <i>Enumeração de códigos de acompanhamento</i>  | Information Disclosure / Spoofing                      | Alto                      | Utilização de códigos com elevada entropia, armazenamento em hash e rate limiting | O TrackingCode é gerado através de SecureRandom, utilizando formato GR-.... O sistema implementa validação de códigos, rate limiting e geração de alertas de segurança após múltiplas tentativas inválidas | Implementado              |
| <i>Upload de ficheiros maliciosos</i>           | Tampering / Denial of Service / Elevation of Privilege | Alto                      | Restrição de tipos de ficheiro, validação de uploads e limitação de tamanho       | O FileStorageService valida MIME types, extensões, tamanho máximo de 10 MB e bloqueia extensões executáveis. Uploads suspeitos geram alertas de segurança                                                  | Parcialmente implementado |
| <i>Path traversal no sistema de ficheiros</i>   | Elevation of Privilege / Information Disclosure        | Alto                      | Canonicalização de paths e utilização de diretórios controlados                   | O SafeFilename e o FileStorageService sanitizam nomes de ficheiros, normalizam caminhos com toAbsolutePath().normalize() e verificam que os paths permanecem dentro da diretoria base da aplicação         | Implementado              |
| <i>Descoberta da identidade do denunciante</i>  | Information Disclosure                                 | Médio /Alto               | Minimização de dados e anonimização                                               | O sistema permite submissão anónima sem criação de conta. Os logs evitam exposição de tracking codes e dados sensíveis, sendo os detalhes sanitizados pelo AuditLogService                                 | Parcialmente implementado |
| <i>Alteração indevida do estado da denúncia</i> | Tampering / Elevation of Privilege                     | Médio                     | RBAC e validação server-side                                                      | O SecurityConfig protege endpoints administrativos e de analistas. Apenas utilizadores autorizados podem alterar estados, prioridades e notas de casos                                                     | Implementado              |
| <i>Eliminação ou modificação de provas</i>      | Tampering / Repudiation                                | Alto                      | Hashing, controlo de acessos e backups                                            | Os anexos e pacotes de evidência utilizam verificação SHA-256, manifests de integridade e diretórios controlados pela aplicação                                                                            | Parcialmente implementado |
| <i>Roubo ou manipulação de sessão</i>           | Spoofing / Tampering                                   | Médio                     | Tokens seguros e autenticação protegida                                           | O sistema utiliza JWT Bearer stateless, BCrypt para passwords e desativação de Basic Authentication                                                                                                        | Parcialmente implementado |
| <i>Enchimento do disco através de uploads</i>   | Denial of Service                                      | Alto                      | Limites de upload e monitorização                                                 | Foram implementados limites de tamanho e rate limiting nos endpoints públicos de upload                                                                                                                    | Parcialmente implementado |

|                                                       |                                                 |             |                                                |                                                                                                                                   |                           |
|-------------------------------------------------------|-------------------------------------------------|-------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <i>Modificação de logs</i>                            | Tampering / Repudiation                         | Médio /Alto | Restrição de acesso e backups                  | Os logs de auditoria encontram-se protegidos através de RBAC e são exportados em backups da aplicação                             | Parcialmente implementado |
| <i>Acesso indevido a ficheiros por IDOR</i>           | Information Disclosure / Elevation of Privilege | Alto        | Verificação de ownership e controlo de acessos | <i>O acompanhamento público da denúncia exige tracking code válido; os downloads internos verificam autenticação e ownership.</i> | Implementado              |
| <i>Compromisso de contas privilegiadas</i>            | Spoofing / Elevation of Privilege               | Alto        | Password hashing, RBAC e logging               | Passwords protegidas com BCrypt, endpoints administrativos protegidos por JWT e segregação de papéis ADMIN, ANALYST e AUDITOR     | Parcialmente implementado |
| <i>Falta de rastreabilidade de operações críticas</i> | Repudiation                                     | Médio       | Audit logs estruturados                        | Operações críticas são registadas através de audit logs e security alerts acessíveis a administradores e auditores                | Implementado              |

Esta tabela demonstra a rastreabilidade entre o threat modeling realizado na Phase 1 e os controlos técnicos implementados no Sprint 1 da Phase 2. As ameaças de maior criticidade, como path traversal, uploads maliciosos e enumeração de tracking codes, foram priorizadas durante a implementação.

Algumas mitigações inicialmente previstas, como malware scanning, quotas de armazenamento, MFA e mecanismos append-only para logs, continuam identificadas como melhorias futuras devido à necessidade de integração adicional ou dependência de componentes externos ao backend atual.

A implementação atual não elimina totalmente todos os riscos identificados durante a Phase 1, mas reduz significativamente a superfície de ataque através da utilização de validação de inputs, RBAC, ownership checks, hashing, geração segura de identificadores, controlo de uploads, audit logs, security alerts e mecanismos de verificação de integridade de evidências e backups.

## 13. Testes Automatizados, Segurança e Cobertura

### 13.1 Objetivo

Nesta Sprint foram implementados testes automatizados com o objetivo de validar, de forma repetível, os principais controlos funcionais e de segurança do GhostReport.

Os testes foram orientados pelas ameaças identificadas no threat modeling e pelos requisitos de segurança definidos para o sistema. Desta forma, as mitigações implementadas não ficam apenas documentadas ao nível do desenho da solução, sendo também verificadas tecnicamente através de testes executáveis.

A estratégia de testes procura validar:

| <b>Área</b>                 | <b>Objetivo</b>                                                             |
|-----------------------------|-----------------------------------------------------------------------------|
| <i>Validação de input</i>   | Rejeitar dados inválidos ou maliciosos antes do processamento               |
| <i>RBAC</i>                 | Garantir que cada perfil apenas acede aos endpoints autorizados             |
| <i>Ownership de casos</i>   | Impedir que analistas acedam ou alterem casos atribuídos a outros analistas |
| <i>Upload seguro</i>        | Validar tamanho, MIME type, extensão, assinatura e nomes dos ficheiros      |
| <i>Path traversal</i>       | Impedir acesso a caminhos fora das diretorias controladas pela aplicação    |
| <i>Rate limiting</i>        | Reduzir abuso de endpoints públicos e enumeração de tracking codes          |
| <i>Audit logging</i>        | Registar operações críticas da aplicação                                    |
| <i>Security alerts</i>      | Detetar comportamentos suspeitos                                            |
| <i>Backups</i>              | Validar geração, manifest, hashes e integridade                             |
| <i>Pacotes de evidência</i> | Validar geração segura e verificação de integridade                         |

## 13.2 Estratégia de Testes

A estratégia de testes do GhostReport foi organizada em duas camadas principais: testes unitários e testes de integração.

Os testes unitários validam componentes específicos da aplicação de forma mais isolada, enquanto os testes de integração verificam fluxos completos envolvendo Spring Boot, camada web, segurança, validação, persistência e serviços aplicacionais.

Esta abordagem permite detetar falhas em diferentes níveis da aplicação: desde regras locais de domínio, como nomes de ficheiros e tracking codes, até fluxos completos como submissão de denúncias, autorização por perfil, criação de backups e geração de pacotes de evidência.

## 13.3 Testes Unitários

Os testes unitários focam-se nas menores unidades verificáveis de comportamento. No GhostReport, estes testes cobrem principalmente primitivas de domínio, validações e serviços com lógica de segurança própria.

| <b>Teste</b>                    | <b>Objetivo</b>                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------|
| <i>SafeFilenameSecurityTest</i> | Verificar rejeição de nomes perigosos, extensões proibidas e tentativas de path traversal |
| <i>FileStorageServiceTest</i>   | Validar regras de upload, MIME types, extensões, magic bytes e armazenamento seguro       |



|                                    |                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------|
| <i>RateLimiterServiceTest</i>      | Confirmar bloqueio após excesso de tentativas                                        |
| <i>DataInitializerDisabledTest</i> | Garantir que utilizadores seed não são criados quando a configuração está desativada |
| <i>TrackingCodeEnumerationTest</i> | Validar comportamento perante tentativas repetidas com códigos inválidos             |

Estes testes incluem cenários positivos e negativos, garantindo que a aplicação aceita entradas válidas e rejeita entradas perigosas ou inconsistentes.

### 13.4 Testes de Integração

Os testes de integração verificam o comportamento da aplicação quando várias camadas funcionam em conjunto. Estes testes exercitam controladores, serviços, validação, autenticação, autorização, persistência e serialização HTTP.

| <b>Teste</b>                                | <b>Objetivo</b>                                                                  |
|---------------------------------------------|----------------------------------------------------------------------------------|
| <i>PublicReportFlowIntegrationTest</i>      | Validar o fluxo público de criação e acompanhamento de denúncias                 |
| <i>ReportControllerAttachmentUploadTest</i> | Validar upload de anexos através do controller                                   |
| <i>AdminAuthorizationTest</i>               | Confirmar que apenas administradores acedem a endpoints administrativos          |
| <i>AuditorAuthorizationTest</i>             | Validar acesso aos endpoints <code>/audit/**</code> por auditor ou administrador |
| <i>RbacAuthorizationMatrixTest</i>          | Verificar a matriz geral de permissões da aplicação                              |
| <i>AnalystCaseOwnershipTest</i>             | Confirmar isolamento entre analistas e casos atribuídos                          |
| <i>ClosedCaseSecurityTest</i>               | Impedir alterações indevidas em casos encerrados                                 |
| <i>AuditLogSecurityTest</i>                 | Validar proteção e consulta de logs de auditoria                                 |
| <i>AnonymousDataLoggingTest</i>             | Garantir minimização de dados sensíveis nos logs                                 |
| <i>AdminBackupControllerSecurityTest</i>    | Validar proteção dos endpoints administrativos de backup                         |
| <i>BackupServiceIntegrationTest</i>         | Testar criação, manifest, hashes e verificação de integridade de backups         |
| <i>CasePackageServiceIntegrationTest</i>    | Validar geração e verificação de pacotes de evidência                            |

### 13.5 Áreas de Segurança Validadas

Os testes automatizados cobrem as principais mitigações implementadas durante a Sprint 1.

| <b>Área de Segurança</b>            | <b>Testes Associados</b>                                                      |
|-------------------------------------|-------------------------------------------------------------------------------|
| <i>RBAC</i>                         | AdminAuthorizationTest, AuditorAuthorizationTest, RbacAuthorizationMatrixTest |
| <i>Ownership de casos</i>           | AnalystCaseOwnershipTest                                                      |
| <i>Casos encerrados</i>             | ClosedCaseSecurityTest                                                        |
| <i>Upload seguro</i>                | ReportControllerAttachmentUploadTest, FileStorageServiceTest                  |
| <i>Path traversal</i>               | SafeFilenameSecurityTest, FileStorageServiceTest                              |
| <i>Enumeração de tracking codes</i> | TrackingCodeEnumerationTest                                                   |
| <i>Rate limiting</i>                | RateLimiterServiceTest                                                        |
| <i>Headers de segurança</i>         | SecurityHeadersTest                                                           |
| <i>Logs de auditoria</i>            | AuditLogSecurityTest, AnonymousDataLoggingTest                                |
| <i>Backups</i>                      | BackupServiceIntegrationTest, AdminBackupControllerSecurityTest               |
| <i>Pacotes de evidência</i>         | CasePackageServiceIntegrationTest                                             |

Esta cobertura demonstra que os principais controlos de segurança implementados foram verificados através de testes automatizados e não apenas descritos documentalmente.

### 13.6 Testes aos Pacotes de Evidência

Foi dada especial atenção à funcionalidade de geração de pacotes de evidência, uma vez que esta envolve operações sensíveis no sistema de ficheiros e está diretamente relacionada com integridade, rastreabilidade e preservação de evidência digital.

Os testes implementados verificam que:

- não é possível gerar um pacote para casos ainda abertos;
- apenas o analista responsável ou um administrador pode gerar o pacote;
- o pacote é gerado apenas para casos RESOLVED ou REJECTED;
- são criados os ficheiros case\_summary.txt, evidence\_manifest.json e integrity\_hashes.txt;
- os anexos são incluídos no pacote com hashes SHA-256;
- a integridade do pacote pode ser verificada posteriormente;
- a operação é registada em audit log.

### 13.7 Execução dos Testes

Os testes foram executados localmente através do Maven Wrapper com o seguinte comando:

```
.\mvnw.cmd test
```

Resultado obtido:

```
[INFO] Results:
[INFO]
[INFO] Tests run: 58, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 39.098 s
[INFO] Finished at: 2026-05-15T17:33:12+01:00
[INFO] -----
```

Figura 2 - Execução dos testes automatizados com Maven Wrapper

Este resultado demonstra que a suite de testes automatizados foi executada com sucesso, sem falhas ou erros.

Além da execução local, os testes são também executados automaticamente no GitHub Actions através do workflow de CI, permitindo validar alterações submetidas ao repositório de forma contínua.

### 13.8 Cobertura com JaCoCo

Para complementar a execução dos testes, foi integrado o JaCoCo como ferramenta de medição de cobertura de código.

O JaCoCo gera relatórios em formato HTML, XML e CSV, permitindo analisar a cobertura por package e identificar áreas com menor validação automatizada.

Localização do relatório HTML gerado localmente:

```
target/site/jacoco/index.html
```

Na pipeline, o relatório é publicado como artefacto do GitHub Actions, permitindo a sua consulta após cada execução.

Resultado de cobertura obtido:

## ghostreport

| Element                                    | Missed Instructions    | Cov. | Missed Branches        | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|--------------------------------------------|------------------------|------|------------------------|------|--------|------|--------|-------|--------|---------|--------|---------|
| <a href="#">com.ghostreport.service</a>    | <div><div></div></div> | 71%  | <div><div></div></div> | 54%  | 161    | 338  | 349    | 1 191 | 32     | 167     | 0      | 19      |
| <a href="#">com.ghostreport.dto</a>        | <div><div></div></div> | 68%  | <div><div></div></div> | n/a  | 49     | 107  | 89     | 182   | 49     | 107     | 5      | 25      |
| <a href="#">com.ghostreport.controller</a> | <div><div></div></div> | 60%  | <div><div></div></div> | 25%  | 17     | 43   | 28     | 80    | 13     | 39      | 0      | 6       |
| <a href="#">com.ghostreport.model</a>      | <div><div></div></div> | 83%  | <div><div></div></div> | 75%  | 19     | 112  | 29     | 165   | 18     | 110     | 0      | 9       |
| <a href="#">com.ghostreport.exception</a>  | <div><div></div></div> | 52%  | <div><div></div></div> | 100% | 3      | 7    | 15     | 31    | 3      | 6       | 0      | 1       |
| <a href="#">com.ghostreport.domain</a>     | <div><div></div></div> | 82%  | <div><div></div></div> | 62%  | 13     | 25   | 6      | 43    | 1      | 9       | 0      | 3       |
| <a href="#">com.ghostreport.config</a>     | <div><div></div></div> | 93%  | <div><div></div></div> | 50%  | 5      | 25   | 5      | 68    | 1      | 21      | 0      | 4       |
| <a href="#">com.ghostreport.security</a>   | <div><div></div></div> | 98%  | <div><div></div></div> | 71%  | 5      | 32   | 2      | 80    | 1      | 25      | 0      | 4       |
| <a href="#">com.ghostreport</a>            | <div><div></div></div> | 37%  | <div><div></div></div> | n/a  | 1      | 2    | 2      | 3     | 1      | 2       | 0      | 1       |
| Total                                      | 1 939 of 7 369         | 73%  | 180 of 402             | 55%  | 273    | 691  | 525    | 1 843 | 119    | 486     | 5      | 72      |

Figura 3 - Relatório criado pelo JaCoCo

A cobertura global obtida demonstra uma base relevante de validação automatizada, especialmente nas áreas de segurança, configuração, domínio, serviços e modelos.

As áreas com menor cobertura, como DTOs, exceções e alguns controladores, representam oportunidades de melhoria para próximas sprints, nomeadamente através de testes adicionais para validação de erros, respostas HTTP e fluxos alternativos.

## Pit Test Coverage Report

### Project Summary

| Number of Classes | Line Coverage                        | Mutation Coverage                  | Test Strength                      |
|-------------------|--------------------------------------|------------------------------------|------------------------------------|
| 49                | 71% <div><div></div></div> 1286/1800 | 56% <div><div></div></div> 371/666 | 73% <div><div></div></div> 371/508 |

### Breakdown by Package

| Name                                       | Number of Classes | Line Coverage                       | Mutation Coverage                  | Test Strength                      |
|--------------------------------------------|-------------------|-------------------------------------|------------------------------------|------------------------------------|
| <a href="#">com.ghostreport.config</a>     | 3                 | 93% <div><div></div></div> 64/69    | 83% <div><div></div></div> 25/30   | 89% <div><div></div></div> 25/28   |
| <a href="#">com.ghostreport.controller</a> | 6                 | 65% <div><div></div></div> 52/80    | 28% <div><div></div></div> 11/40   | 46% <div><div></div></div> 11/24   |
| <a href="#">com.ghostreport.domain</a>     | 2                 | 89% <div><div></div></div> 24/27    | 64% <div><div></div></div> 9/14    | 69% <div><div></div></div> 9/13    |
| <a href="#">com.ghostreport.dto</a>        | 15                | 48% <div><div></div></div> 83/172   | 43% <div><div></div></div> 19/44   | 61% <div><div></div></div> 19/31   |
| <a href="#">com.ghostreport.exception</a>  | 1                 | 52% <div><div></div></div> 16/31    | 40% <div><div></div></div> 2/5     | 100% <div><div></div></div> 2/2    |
| <a href="#">com.ghostreport.model</a>      | 6                 | 81% <div><div></div></div> 121/150  | 54% <div><div></div></div> 28/52   | 58% <div><div></div></div> 28/48   |
| <a href="#">com.ghostreport.security</a>   | 4                 | 95% <div><div></div></div> 78/82    | 70% <div><div></div></div> 19/27   | 76% <div><div></div></div> 19/25   |
| <a href="#">com.ghostreport.service</a>    | 12                | 71% <div><div></div></div> 848/1189 | 57% <div><div></div></div> 258/454 | 77% <div><div></div></div> 258/337 |

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.23.1 [support](#)

Enhanced functionality available at [arcmutate.com](#)

Figura 4 - Relatório criado pelo Pit

O PIT gerou 666 mutações, das quais 371 foram eliminadas pelos testes automatizados, correspondendo a uma mutation coverage de 56% e uma line coverage de 71%. O relatório identifica também os pacotes com menor capacidade de detecção de mutações, permitindo orientar futuras melhorias na qualidade dos testes.

## 13.9 Execução em Pipeline

A pipeline de CI executa automaticamente os testes e gera os relatórios de cobertura. O workflow prepara o ambiente Java 17, configura cache Maven, disponibiliza PostgreSQL 16 como service container e executa os comandos necessários para validar a aplicação.

Comandos principais executados pela pipeline:

`./mvnw clean compile`

`./mvnw test jacoco:report`

Artefactos produzidos:

| <b>Artefacto</b>              | <b>Conteúdo</b>                                      |
|-------------------------------|------------------------------------------------------|
| <i>surefire-test-reports</i>  | Relatórios de execução dos testes                    |
| <i>jacoco-coverage-report</i> | Relatório de cobertura HTML/XML/CSV e ficheiro .exec |

A publicação destes artefactos permite manter evidência técnica da execução dos testes e da cobertura obtida em ambiente de integração contínua.

## 14. Conclusão

A Sprint 1 da Phase 2 permitiu transformar o trabalho de análise e desenho realizado na Phase 1 numa implementação funcional e verificável do GhostReport. Enquanto a Phase 1 se focou na identificação de requisitos, ativos, ameaças, abuse cases, STRIDE, attack trees e mitigações planeadas, esta sprint concretizou vários desses controlos ao nível do código, dos testes e da pipeline DevSecOps.

Durante esta fase foram implementados mecanismos centrais de segurança, incluindo autenticação JWT stateless, hashing de passwords com BCrypt, controlo de acessos baseado em papéis, regras de ownership para casos atribuídos, validação de inputs, proteção contra path traversal, validação segura de uploads, tracking codes com elevada entropia, rate limiting, audit logs, security alerts, backups com verificação de integridade e pacotes de evidência com hashes SHA-256.

A implementação foi acompanhada por testes automatizados orientados aos principais riscos identificados na Phase 1. Estes testes validam controlos como RBAC, autorização por perfil, isolamento entre analistas, uploads seguros, enumeração de tracking codes, logs de auditoria, backups e pacotes de evidência. A integração do JaCoCo permitiu ainda medir a cobertura da suite de testes, reforçando a capacidade de avaliação objetiva da qualidade dos testes implementados.

A pipeline DevSecOps criada em GitHub Actions introduz validação contínua através de build, testes, análise SAST com SpotBugs, análise SCA com OWASP Dependency-Check, secret scanning com Gitleaks, DAST com OWASP ZAP Baseline e publicação de artefactos de segurança. Esta automatização melhora a rastreabilidade da entrega e aproxima o projeto dos princípios de SSDLC definidos para a unidade curricular.

Apesar das limitações ainda existentes, como a ausência de malware scanning real, MFA, rate limiting distribuído e logs tamper-proof, a Sprint 1 estabelece uma base sólida para evolução futura. O sistema já demonstra uma aplicação prática de defesa em profundidade, menor privilégio, validação server-side, monitorização e verificação automatizada de controlos de segurança.

Conclui-se que o GhostReport evoluiu de um modelo conceptual seguro para uma aplicação funcional com mecanismos concretos de segurança, testes executáveis e evidência técnica gerada por pipeline. Esta sprint cumpre o objetivo de demonstrar a implementação inicial de práticas de desenvolvimento seguro e cria uma base consistente para hardening e expansão nas próximas fases do projeto.